

GMD2

Lecture 08: Zelda

The Legend of Zelda



Today's Goal

The Legend of VIA

Press Enter

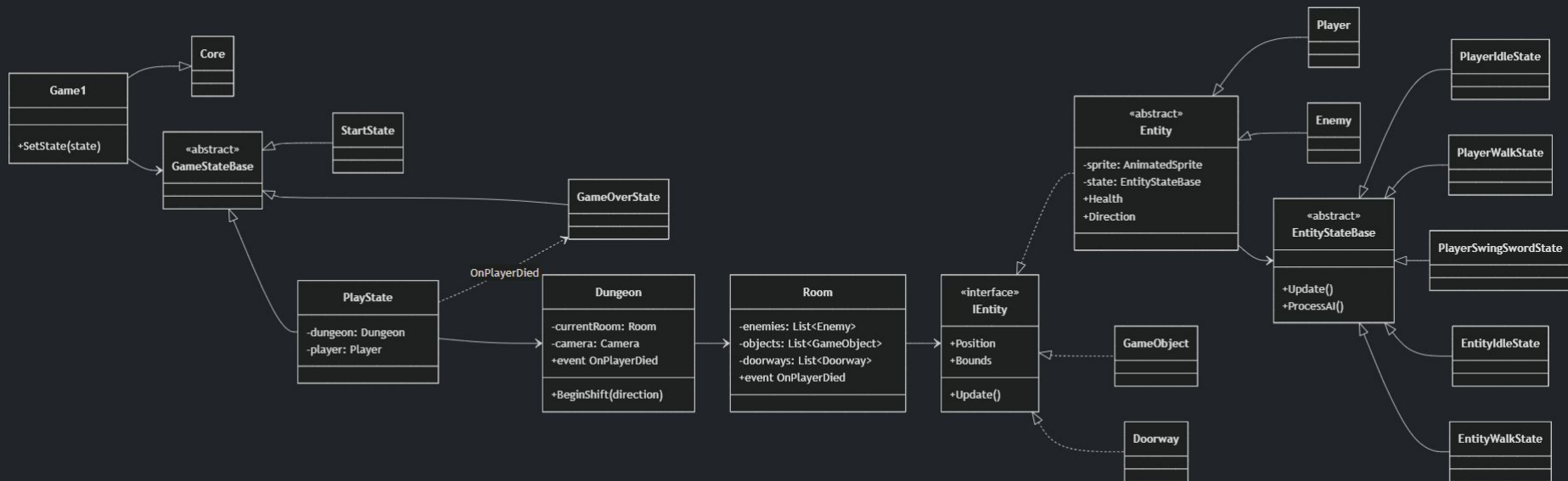
Today's Topics

- Assets and Tooling
- Dungeon Generation
- Hitboxes/Hurtboxes
- Observer Pattern (Delegates & Events)
- Screen Scrolling
- Data Driven Design

Source Code

- All Source code for today's game can be found here:
 - <https://github.com/Metamate/gmd2-zelda>
- Walkthrough in README.md

Diagram



Explore the Codebase

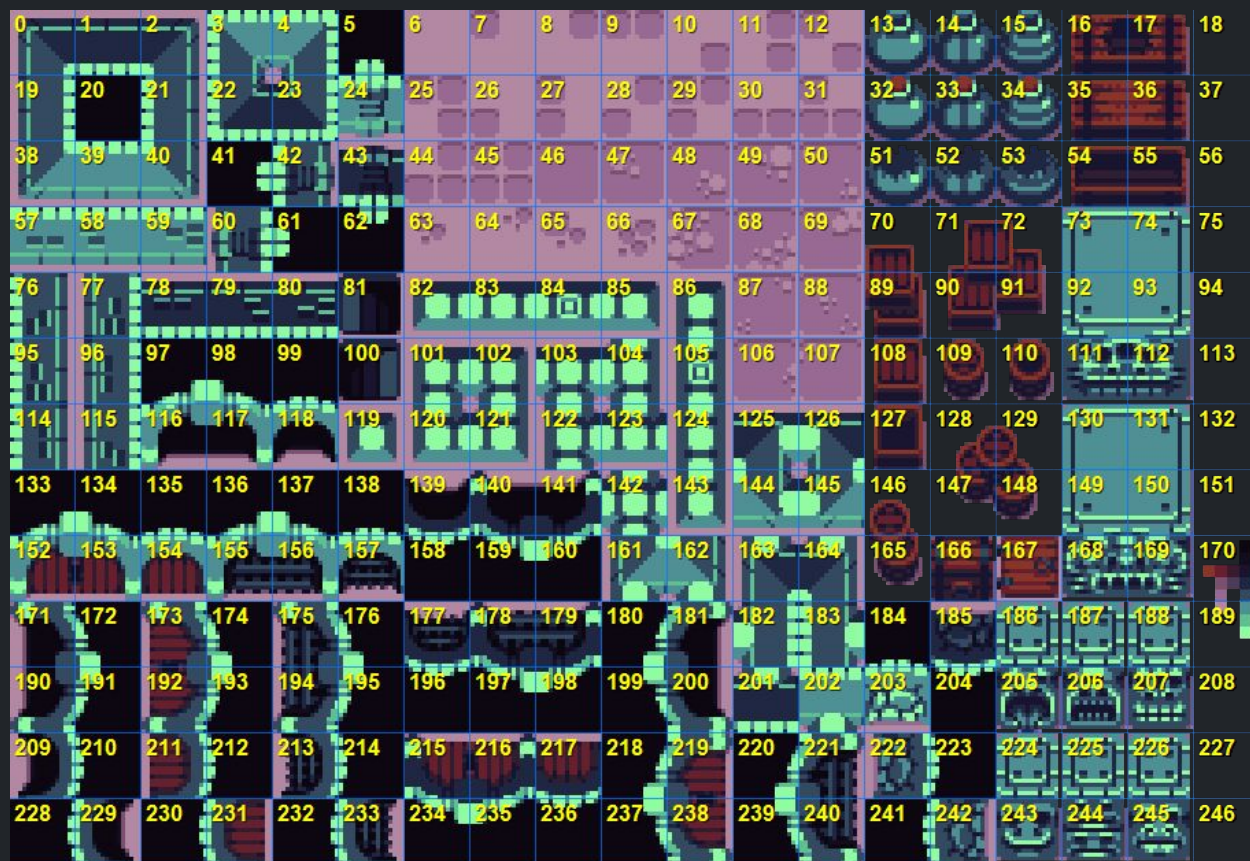
Take ~15 minutes to browse the code. Use the README as a guide if you need it.

- Find where the game transitions between the title screen, gameplay, and game over
- How does the player move?
- Find where a new room is generated and what it contains when it is created
- The player and enemies share a lot of code
 - Find the common base and explain what it provides
- Trace what happens from the moment the player presses the spacebar to an enemy taking damage

Assets



Tooling

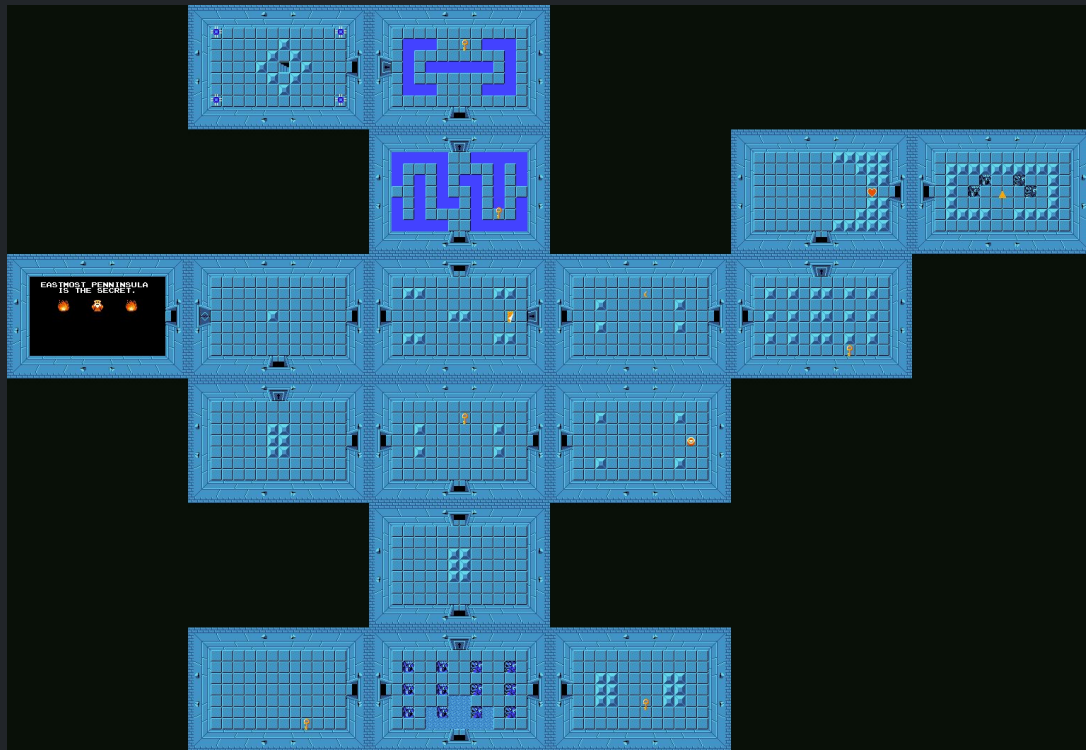


Top-Down Perspective



Dungeon Generation

Dungeon Maps



Dungeon Generation

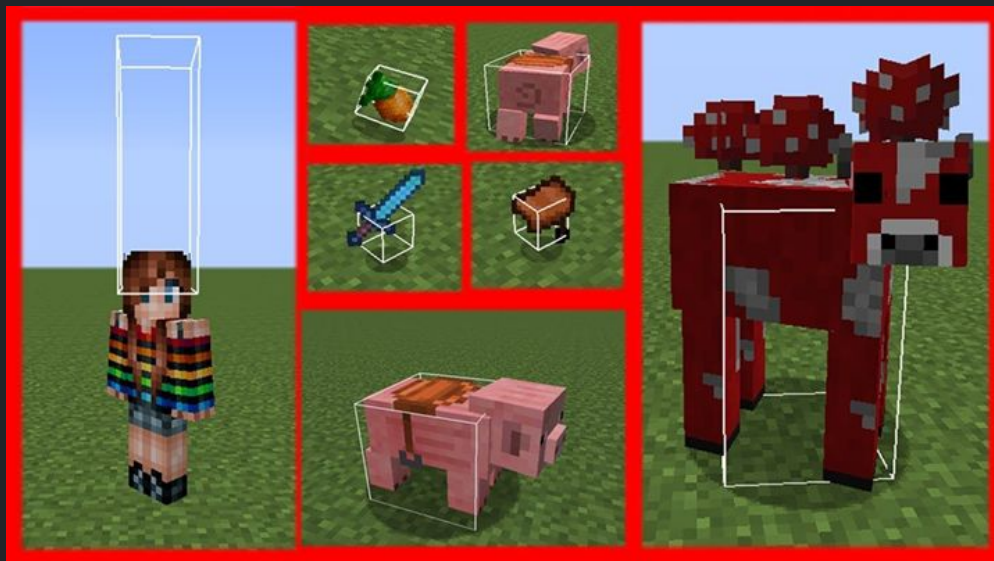


Hitboxes/Hurtboxes



Hitbox

Hurtbox

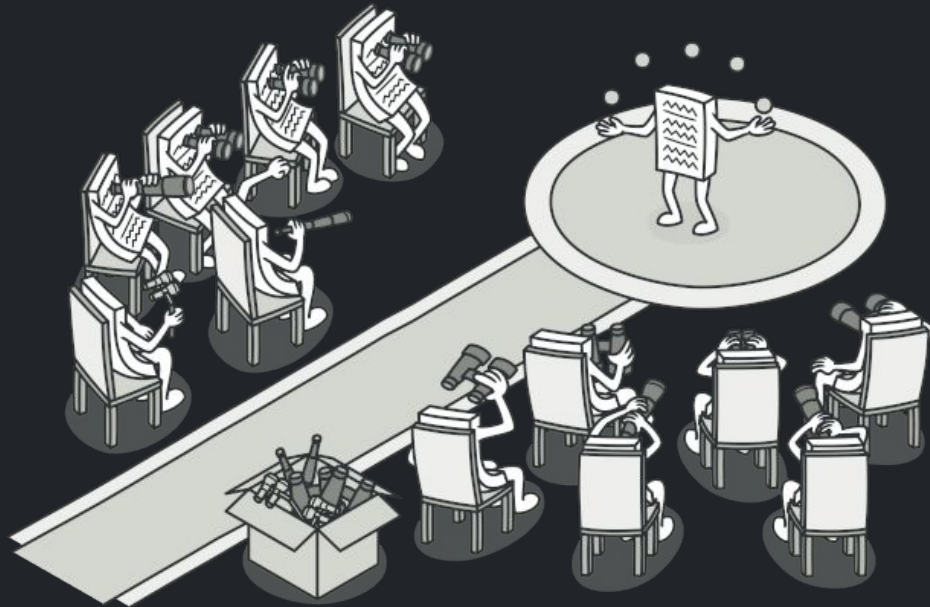


Events in Games

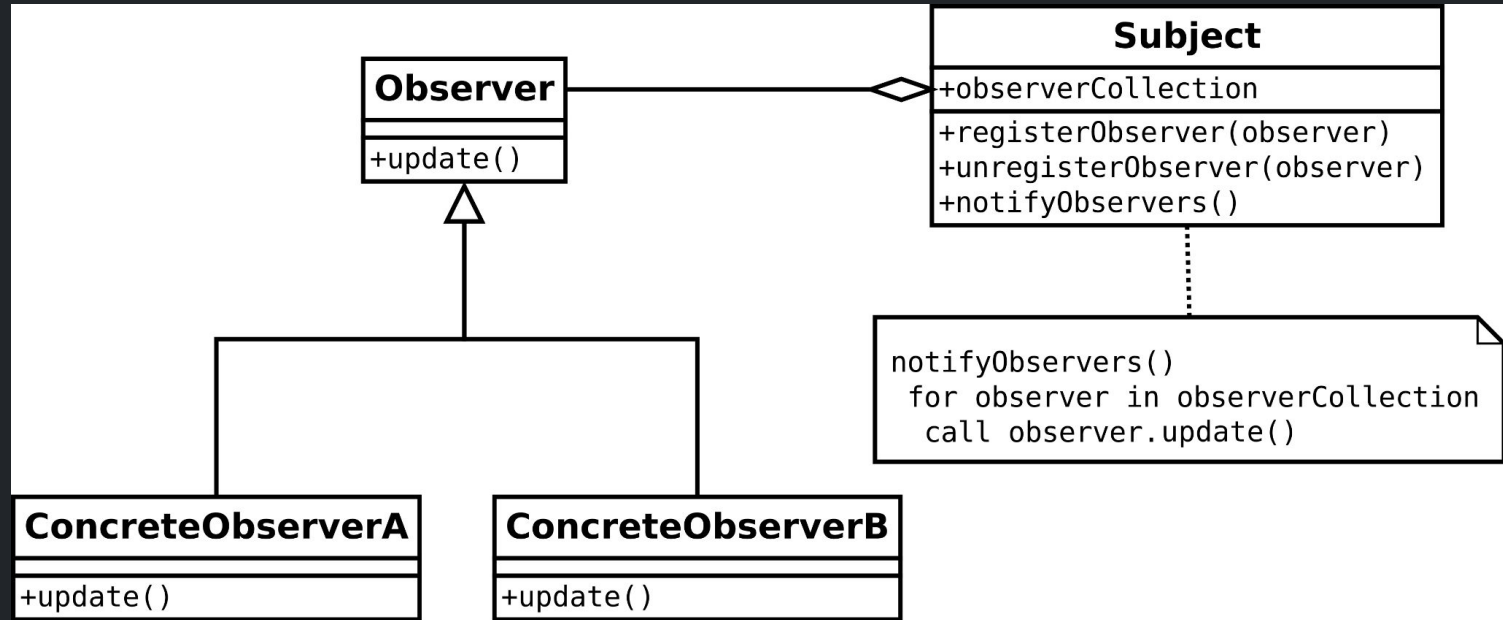
- Games are full of moments where something happens and other things need to react
- The thing that happened shouldn't need to know what those other things are
- Events let you flip that around: the trigger just announces, anything that cares subscribes

Something happens	Things that might react
Player enters a room	Lock the doors, spawn enemies, start music
Player dies	Show game over screen, save score, update stats
Floor switch pressed	Open doors, play sound, update minimap
Enemy killed	Drop loot, increment kill counter, trigger achievement
Sword swing lands	Deal damage, play hit sound, spawn particles

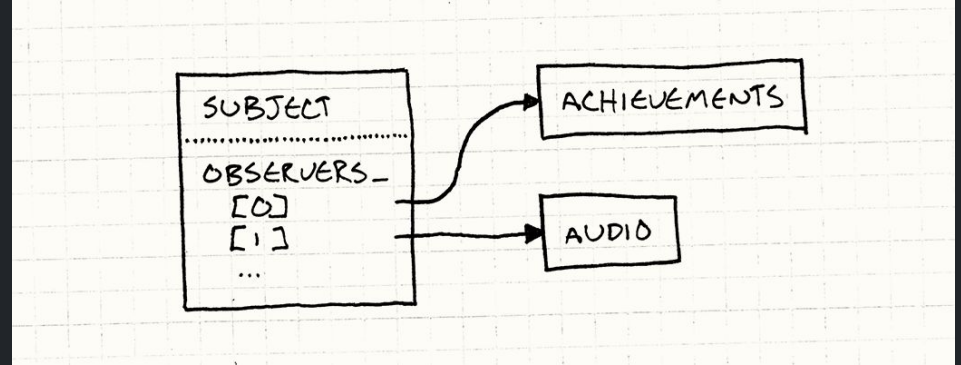
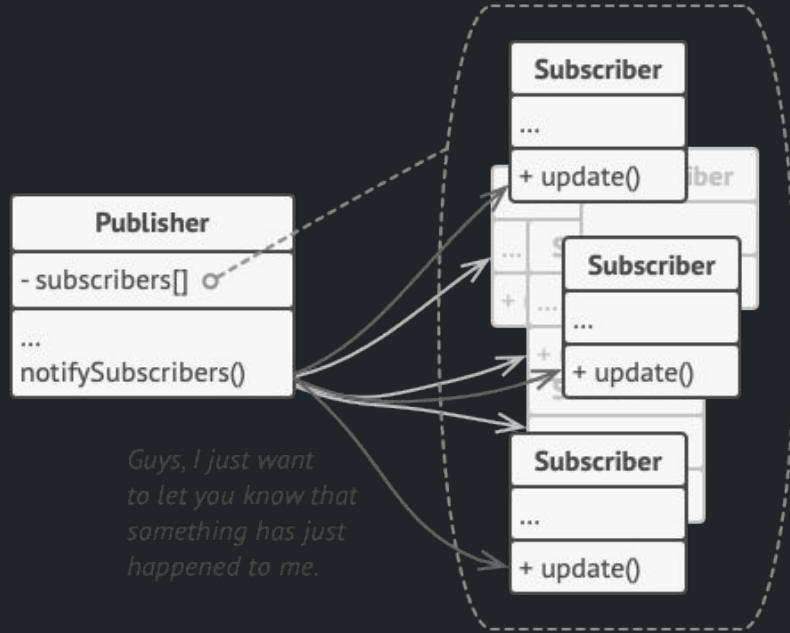
Observer Pattern



Observer Pattern



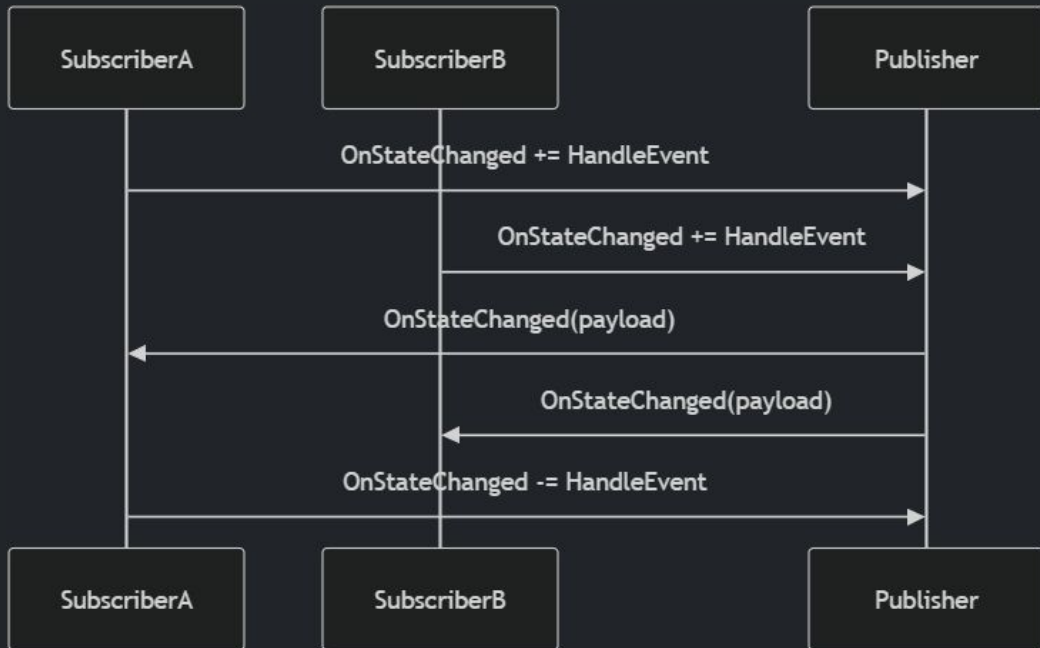
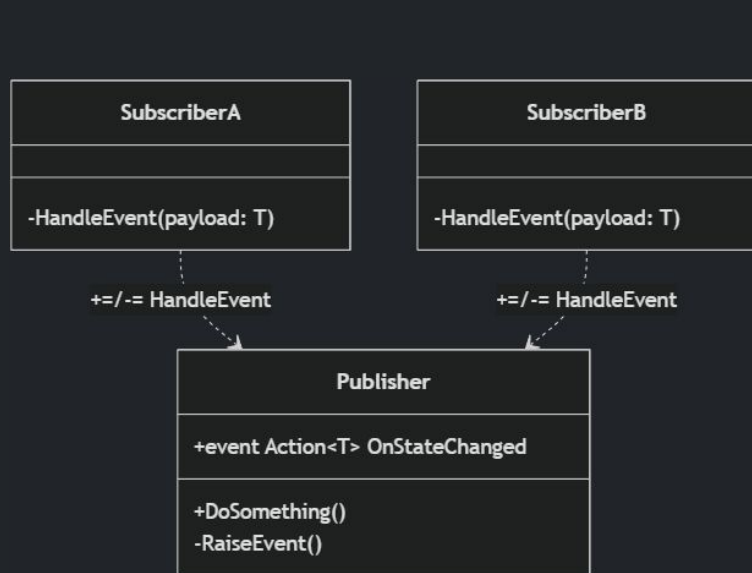
Publisher-Subscriber / Subject-Observer



However...

In C#, we don't need to implement the observer pattern. Events are part of the language!

C# Events



Delegates

A delegate is a variable that holds a reference to a method — call it later, from anywhere.

```
// Built-in delegate types:
```

```
Action           // returns void, no args
```

```
Action<T>        // returns void, takes T
```

```
Func<T>          // returns T, no args
```

```
// Assign one handler
```

```
obj.OnCollide = () => Console.WriteLine("hit");
```

```
// Add multiple handlers
```

```
obj.OnCollide += () => PlaySound("hit");
```

```
obj.OnCollide += () => FlashRed();
```

```
// Call all subscribers
```

```
OnCollide?.Invoke();
```

```
// GameObject.cs
```

```
public Action OnCollide { get; set; }
```

```
// Room.cs
```

```
switchObj.OnCollide += () => {  
    switchObj.State = "pressed";  
    foreach (var d in Doorways) d.IsOpen = true;  
};
```

C# Events

- An event is a delegate with guardrails
 - under the hood it is a delegate, but the event keyword restricts what outside code can do with it

	<code>public Action OnCollision</code>	<code>public event Action OnCollision</code>
Subscribe	<code>obj.OnCollision += handler</code>	<code>obj.OnCollision += handler</code>
Unsubscribe	<code>obj.OnCollision -= handler</code>	<code>obj.OnCollision -= handler</code>
Overwrite	<code>obj.OnCollision = handler</code> ✓	<code>obj.OnCollision = handler</code> ✗ compile error
Invoke from outside	<code>obj.OnCollision()</code> ✓	<code>obj.OnCollision()</code> ✗ compile error
Invoke from inside	<code>OnCollision()</code> ✓	<code>OnCollision()</code> ✓

Lambda Expressions

(parameters) => expression

- A lambda is an anonymous function
 - A function defined inline, without a name
- A short piece of behaviour in one place without declaring a whole method for it
- Implicit return
- Common use: passing behaviour as an argument, or subscribing to events

```
bool IsEven(int n) { return n % 2 == 0; }  
  
// The same thing as a lambda  
(int n) => n % 2 == 0
```

Lambda Expressions & Events

```
switchObj.OnCollide += () =>
{
    if (switchObj.State == "unpressed")
    {
        switchObj.State = "pressed";
        foreach (var d in Doorways)
            d.IsOpen = true;
    }
};
```

- The lambda captures variables from its surrounding scope
 - E.g. switchObj & Doorways. This is called a closure.
- Without lambdas, the switch behaviour would need to be a separate named method that somehow still had access to switchObj and Doorways

Making Zelda (more) Event-driven

- OnPlayerDied is detected in one place but causes a game over screen to appear somewhere completely different. Trace the full chain through the code.
- Enemy death is handled asymmetrically to player death.
 - What happens now when an enemy dies?
 - What could happen if there were an event for it?
 - What would you need to add, and where?
- SoundManager.PlaySound(...) is called directly from several unrelated classes.
 - Find all the call sites. Why is this a problem, and how would events fix it?
- Pick one of the above and refactor it to be event-driven.
 - The class firing the event should have no reference to the class reacting to it.

Screen Scrolling



Tweening

- Short for “in-betweening”
 - animating a value from A to B over time
- Comes from traditional animation
 - artists drew keyframes, assistants filled in the frames in between
- Instead of snapping a position instantly, you move it a little each frame until it arrives
- Used everywhere in games
 - UI animations, camera movement, character transitions, screen wipes
- The two things you always need to define:
 - What is moving
 - How fast / by what curve ([Easings.net](https://easings.net))

Linear Interpolation

- The simplest tween curve - constant speed from A to B
- $\text{lerp}(a, b, t) = a + (b - a) * t$ where t goes from 0 to 1
 - $t = 0 \rightarrow$ result is a
 - $t = 0.5 \rightarrow$ result is exactly halfway
 - $t = 1 \rightarrow$ result is b
- "Linear" means equal steps
 - The value moves the same amount every frame
- Works on anything numeric
 - floats, positions, colours, even angles
- Non-linear curves (ease-in, ease-out) feel more natural but lerp is a good starting point

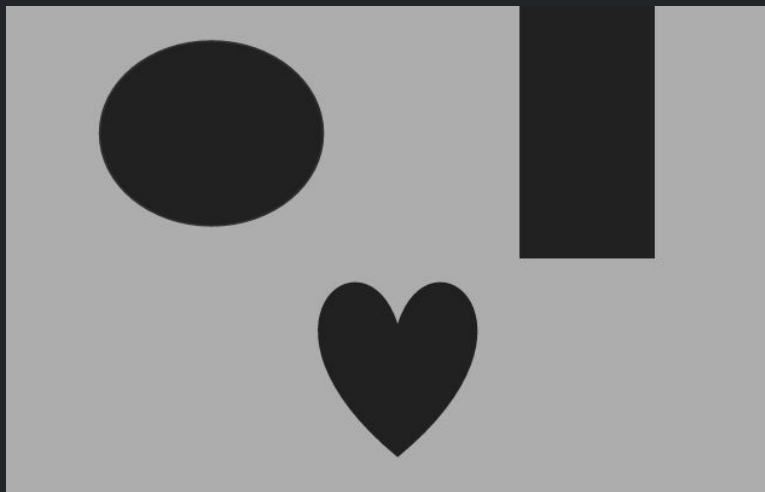
Screen Scrolling

- When the player walks through a door, two things tween simultaneously
 - The camera - from (0, 0) to `_shiftTarget` (one screen width/height away)
 - The player - from the exit doorway to the matching entry point in the new room
- The new room is placed at `_shiftTarget` and never moves
 - The camera travelling toward it is what creates the scroll effect
- All objects render through the same `camera.Transform * screenScaleMatrix`
 - The whole scene shifts together
- When shift is done new room becomes current room
 - Camera & Player position is then reset
 - Gameplay resumes (`Room.Update` is now called again)

```
// Dungeon.cs:147-155
_shiftProgress = Math.Min(1f, _shiftProgress + dt / GameSettings.RoomShiftDuration);
_camera.Position = Vector2.Lerp(Vector2.Zero, _shiftTarget, _shiftProgress);
_player.Position = Vector2.Lerp(_shiftPlayerStart, _shiftPlayerEnd, _shiftProgress);
```

Stenciling

[Stencil buffer](#)



Data Driven Design

- Content lives in config files, behaviour lives in C#
 - This project uses XML — alternatives: JSON, YAML, TOML, Lua
- Definition classes parse the files once at startup into dictionaries
 - The rest of the code never touches XML
- Adding a new enemy type = one <Enemy> block in XML + a spritesheet row
 - No C# changes, no recompile
- GameObject has no if (type == "switch") anywhere
 - the XML defines valid states, the C# just looks up the current state name at draw time

```
<Enemy type="skeleton" width="16" height="16" walkSpeed="20" health="1">  
  <Animation name="walk-down" frames="9,10,11,10" interval="0.2" />  
  <Animation name="idle-down" frames="10" interval="0.2" />  
</Enemy>
```

Tradeoff: typos in XML become runtime errors, not compile errors — the compiler can't catch type="skelton"

Extending Zelda

- Make some enemies drop hearts randomly, which heal the player for 2 damage
 - I.e. one whole heart
- Allow the player to lift pots
 - Using the animations included in the sprite sheets [here](#)
- Pots should stick to the player while they are carrying them
 - The pot needs to track player position
- Allow the player to throw pots and damage enemies
 - If it hits a wall, an enemy, or travels farther than four tiles, destroy it.

Apply it to Your Project

- What are the meaningful moments in your game?
 - The things that happen that other systems might care about
- Map out at least two of them
 - What fires the event, and what should react?
- Is there anywhere a class knows too much about another?
 - Could an event fix that?
- If you haven't started yet: design your event structure before you write the code.
 - It will shape how your classes relate to each other.

